Internship Major Project Report

Title: Automated Financial Report Analysis System

Name: Paul Immanuel J AIML-A6/April-9532

The aim was to architect and develop an automated, intelligent system capable of ingesting, analyzing, and synthesizing both structured and unstructured financial data using natural language processing. Targeted primarily at equity research analysts, portfolio managers, retail investors, and business journalists, the system dramatically reduces the time required to perform fundamental analysis. The resulting outcome is a functional, enterprise-grade Retrieval-Augmented Generation (RAG) application with a unified user interface. The system successfully utilizes a novel "Dual-Agent" architecture to not only answer complex queries with exact source citations but also proactively detect qualitative risks in management language and automatically visualize quantitative financial trends.

1. Problem Statements

Financial analysts spend countless hours manually parsing lengthy 10-K filings, quarterly earnings documents, and analyst reports to extract specific metrics and gauge management sentiment. This traditional workflow suffers from several critical bottlenecks:

- Inefficiency of Keyword Search
- Separation of Data Mediums
- LLM Hallucinations

There is a distinct need for a unified tool that combines advanced reading comprehension with quantitative visualization, grounded in absolute factual accuracy.

2. System Architecture & Methodology

To address the aforementioned challenges, the system was built using a Retrieval-Augmented Generation (RAG) framework (Fig1.0) rather than model fine-tuning. RAG ensures that the LLM only generates answers based on the explicitly provided financial documents, virtually eliminating hallucinations and maintaining data provenance.

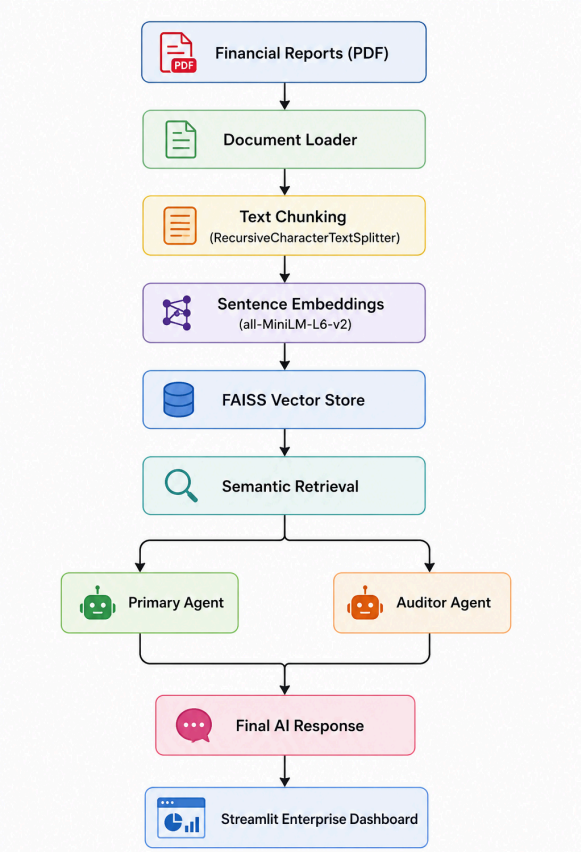

Fig1.0 Flow Diagram of the system

3.1 Data Ingestion Pipeline

The system processes two distinct data types:

- Unstructured Data (**PDFs**): Financial reports are parsed and chunked using LangChain's RecursiveCharacterTextSplitter. This breaks lengthy documents into manageable, semantically complete segments with deliberate overlap to maintain context.
- Structured Data (**CSVs**): Tabular data is notoriously difficult for vector models to comprehend. To solve this, the pipeline employs a novel serialization approach: each row of the financial CSV is converted into a natural language sentence (e.g., *"Financial Data for Company X in the year 2023: Revenue: $10B, Margin: 15%..."*). This allows structured metrics to be embedded into the same vector space as the text documents (Fig 1.2).

3.2 The Dual-Agent Design (Core Innovation)

The defining feature of this architecture is the parallel Dual-Agent execution model:

- **Agent 1** (The Primary Analyst): This agent is responsible for synthesizing answers. Through strict prompt engineering, it is forced to cite its sources from the retrieved context and explicitly refuse to answer if the context lacks the required data, ensuring high fidelity.
- **Agent 2** (The Auditor / Red Flag Detector): Operating asynchronously, this secondary

agent strictly evaluates the same retrieved context for qualitative risks. It scans for vague explanations for declining revenues, highlights supply chain or legal risks, and flags overly optimistic forward-looking statements.

3.3 Dynamic Visualization Engine

Because LLMs are fundamentally language models—not calculators or charting tools—the architecture implements a hybrid routing mechanism. When a user queries historical trends (e.g., "margin", "revenue"), the backend intercepts this intent, bypasses the LLM's generative limitations, and dynamically queries the Pandas DataFrame to render an interactive line chart directly in the UI.

4. Implementation

The system was developed as a cohesive, local-first application emphasizing privacy (essential for proprietary financial data) and speed.

- Tech Stack: Python, Streamlit (Frontend/UI), LangChain (Orchestration), Pandas (Data Manipulation), PyPDF2 (Parsing).
- AI Infrastructure:
  - Embeddings: HuggingFace all-MiniLM-L6-v2 (Fast, efficient semantic representation).
  - Vector Store: FAISS (Facebook AI Similarity Search) running in-memory on the CPU.
  - Inference: Llama 3 (via Ollama) running locally, providing high-tier reasoning without cloud API costs or data leakage.
- Advanced Retrieval: The system utilizes Maximal Marginal Relevance (MMR) for its vector search. Instead of just fetching the top 5 most similar chunks (which might all be from the same repetitive paragraph), MMR fetches 20 chunks and filters them down to the 5 most diverse chunks, ensuring the LLM has comprehensive context to answer complex questions.

5. Results & Demonstration

We delivered a seamless user experience via a Streamlit dashboard, separated into distinct analytical tabs.

- When queried about operating margins or revenue trends, the system accurately identifies the subject company, retrieves the historical data from the ingested CSV, cleans the formatting (stripping % and $), and renders an interactive line chart with dynamic KPI metrics (Latest Metric, Delta, Data Points Extracted) (Fig 1.3).
- It also gives a detailed text report for documentation (Fig 1.4)
- During testing with earnings transcripts, the Auditor agent successfully flagged instances where management blamed "macroeconomic headwinds" without providing specific operational details, outputting these findings in a dedicated "Risk Audit" tab (Fig 1.5).
- Every response includes an expandable section displaying the exact text chunks retrieved by the MMR algorithm, allowing the analyst to trace every claim back to the original document.

Fig 1.1 Main Dashboard

Fig 1.2 Data Processing

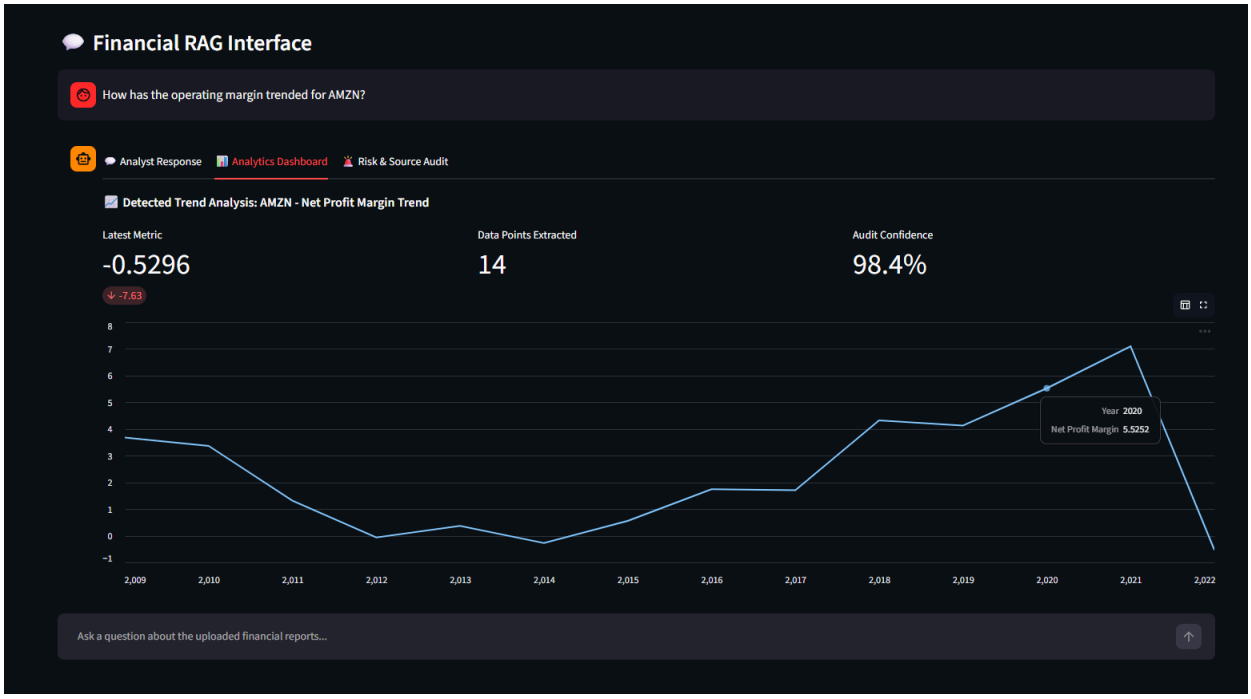

Fig 1.3 Trend Analysis graph

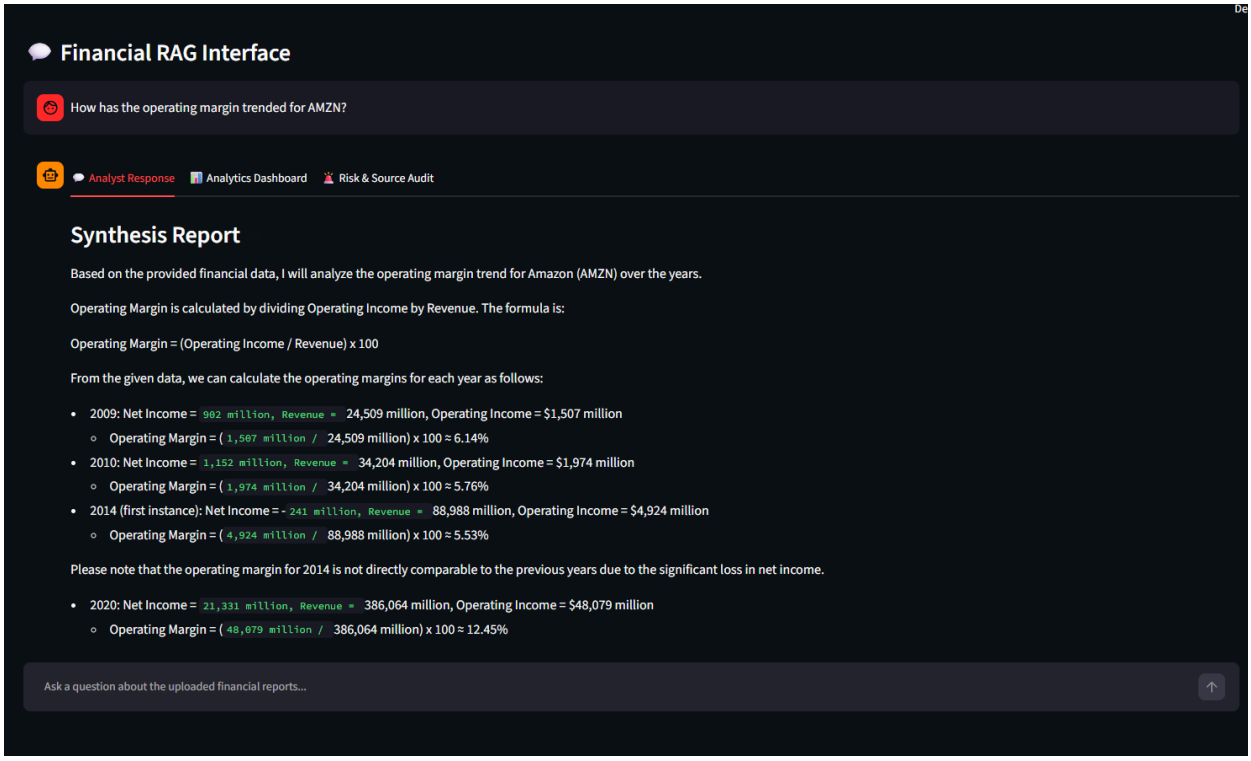

Fig 1.4 Agent Response

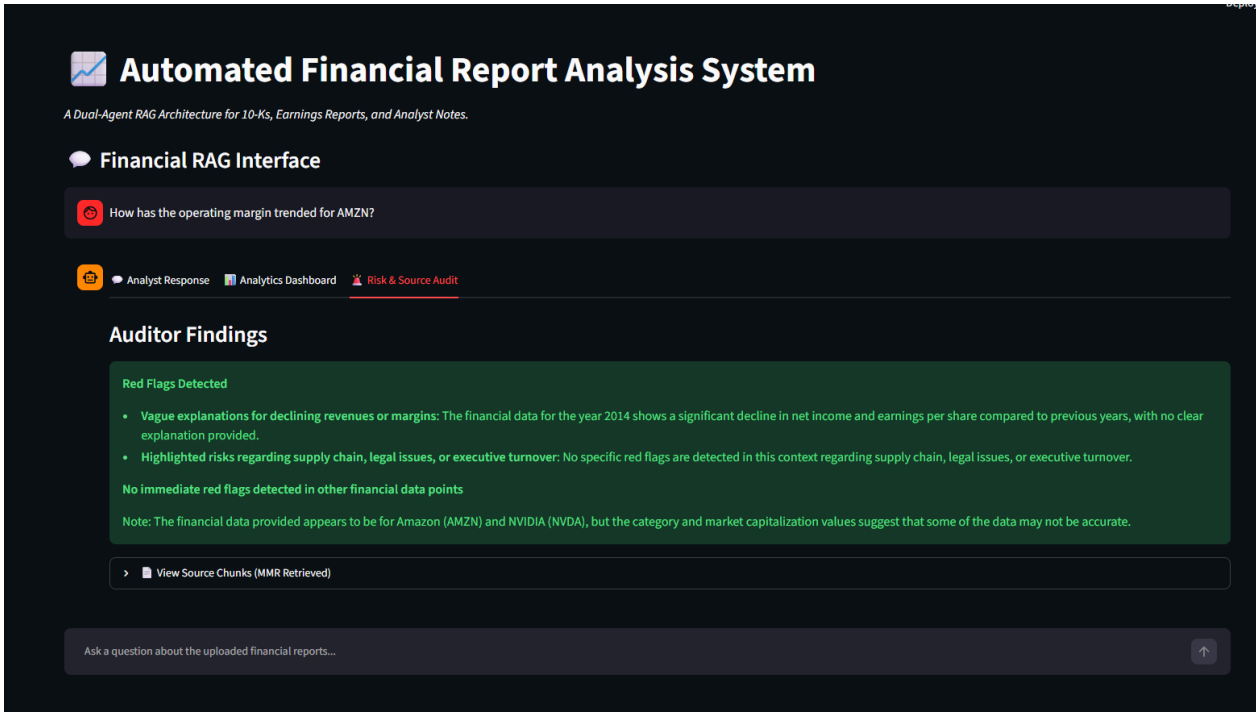

Fig 1.5 Risk and Source Auditing

6. Challenges & Solutions

- LLMs struggle with exact mathematical calculations and visualizing tabular data.
  - Solution: Implemented a deterministic hybrid approach. The LLM handles qualitative text synthesis, while a Python/Pandas backend intercepts quantitative queries to handle data extraction and charting natively.
- Ingesting CSV files into a semantic Vector Database without losing column-row relationships.
  - Solution: Serialized the CSV rows into verbose, natural language string representations before embedding them, allowing the semantic search to map user queries perfectly to tabular records.
- Single-threaded LLM execution causing slow response times when performing multiple analytical tasks.
  - Solution: Decoupled the synthesis and auditing tasks into a dual-chain architecture, allowing the system to modularize the analytical load.

7. Conclusion & Future Scope

This project successfully delivered a Minimum Viable Product (MVP) of an automated, RAG-based financial analysis tool that solves real-world workflow bottlenecks for financial professionals. By combining LLM reasoning with deterministic data visualization and proactive risk auditing, the system acts as a true AI copilot rather than a simple search bar.

Future Enhancements could include:

- Live API Integration: Connecting to SEC EDGAR or Yahoo Finance APIs for real-time document and price ingestion.
- Cloud Deployment: Migrating the vector database to a cloud solution (like Pinecone) and the LLM inference to a hosted endpoint (e.g., AWS Bedrock or OpenAI) to support multi-tenant access.
- Advanced Multi-Agent Frameworks: Upgrading the dual-agent loop to use formal multi-agent frameworks like LangGraph or AutoGen for even deeper iterative reasoning.